CSC 212: Data Structures and Abstractions

11: Recursion

Prof. Marco Alvarez

Department of Computer Science and Statistics
University of Rhode Island

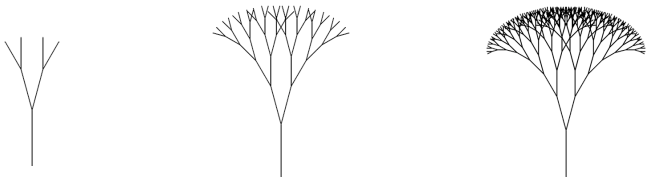
Spring 2026

Recursion

- Definition
  - method of solving problems that involves breaking a problem into smaller and smaller subproblems (of the same structure) until reaching a small enough problem that can be solved trivially
- Recursive functions
  - technically, a recursive function is one that invokes itself
  - must contain at least one base case and one recursive call
    - **base case**: a terminating condition that halts the recursion
    - **recursive case**: a condition that perpetuates the recursion by calling the function again

Why recursion?

- Can we live without it?
  - yes, every recursive algorithm has an equivalent iterative solution
- However …
  - some formulas are inherently recursive in nature
  - some problems naturally lend themselves to recursive solutions



https://courses.cs.washington.edu/courses/cse120/17sp/labs/11/tree.html

Example (python)

```python
import math
import matplotlib.pyplot as plt

def draw(x, y, length, angle, depth):
    if depth == 0:
        return

    end_x = x + length * math.cos(angle)
    end_y = y + length * math.sin(angle)

    plt.plot([x, end_x], [y, end_y])

    draw(end_x, end_y, length * 0.7, angle + 0.5, depth - 1)
    draw(end_x, end_y, length * 0.7, angle - 0.5, depth - 1)

# draw the fractal tree starting from the base
draw(x=0, y=0, length=1, angle=math.pi/2, depth=9)

# remove axes and keep aspect ratio square
plt.axis('off')
plt.axis('equal')
plt.show()
```

Practice

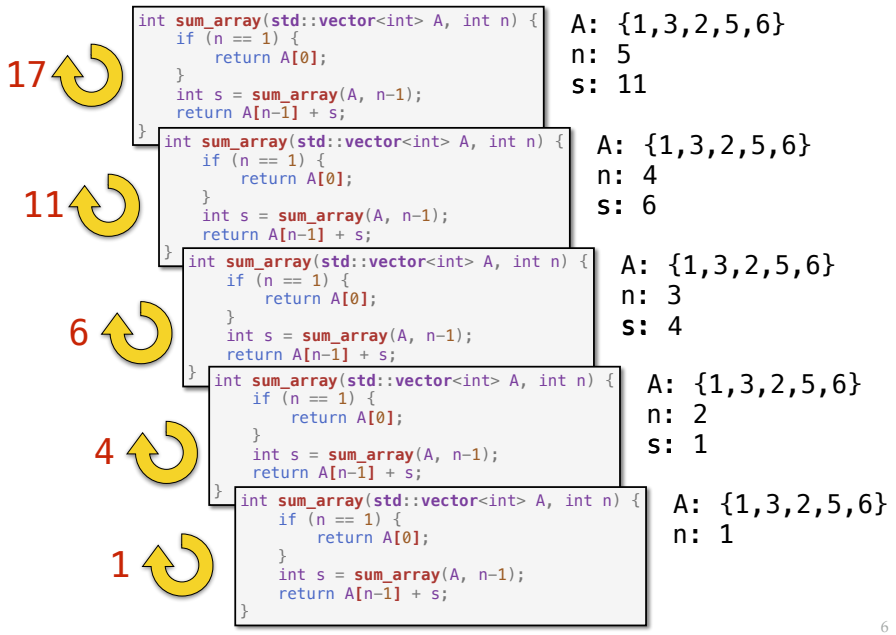
- Write a recursive function to add all elements in a vector
  - ✓ assume that $n > 0$
  - ✓ trace the call sequence with an input array {1,3,2,5,6}, including the parameters passed at each step

```
int sum_array(std::vector<int> A, int n) {
    // base case
    if (n == 1) {
        return A[0];
    }

    // recursive call
    int s = sum_array(A, n-1);

    // return sum
    return A[n-1] + s;
}
```

note that A is passed **by value** (inefficient, but useful for this particular example)



Practice

- Write recursive implementations for the following **singly-linked list** methods
  - ✓ print()
  - ✓ clear()
  - ✓ search(value)
  - ✓ at(index)
  - ✓ reverse()
    - traverses the list and reverses the direction of all node pointers
    - swaps the head and tail pointers

Some recursive methods need **helper (wrapper) functions** to maintain a clean public interface while the helper handles the extra parameters that recursion requires internally

Recursion call tree

- Definition
  - ✓ a **tree** structure that represents the recursive calls of a function
- Properties
  - ✓ the **root** of the tree represents the initial function call
  - ✓ each **node** in the tree represents a distinct function call
  - ✓ the **children** of a node represent the recursive calls invoked by that function call
  - ✓ the **leaves** of the tree represent the base cases
  - ✓ the **height** of the tree represents the maximum depth of the recursion
  - ✓ the **number of nodes** in the tree is equivalent to the total number of calls made

Practice

- Draw the recursion call tree
  - ✓ label each node with local computational cost (number of additions)
  - ✓ express total computational cost as the sum of local computational costs

note that A is now correctly **passed by reference**

```cpp
int sum_array(std::vector<int>& A, int n) {
    if (n == 1) {
        return A[0];
    }
    int partial_sum = sum_array(A, n-1);
    return A[n-1] + partial_sum;
}

int main() {
    std::vector<int> A = {1, 2, 3, 4, 5};
    int sum = sum_array(A, A.size());
    std::cout << "Sum of array: " << sum << std::endl;
    return 0;
}
```


Practice

- Draw the recursion call tree
  - ✓ label each node with local computational cost (number of multiplications)
  - ✓ express total computational cost as the sum of local computational costs

```cpp
double power(double b, int n) {
    if (n == 0) {
        return 1;
    }
   return b * power(b, n-1);
}

int main() {
    std::cout << "3^4: " << power(3, 4) << std::endl;
    return 0;
}
```


Practice

- Draw the recursion call tree
  - ✓ label each node with local computational cost (number of multiplications)
  - ✓ express total computational cost as the sum of local computational costs

```cpp
double power_2(double b, int n) {
    if (n == 0) {
        return 1;
    }
    double half = power_2(b, n/2);
    if (n % 2 == 0) {
        return half * half;
    } else {
        return b * half * half;
    }
}

int main() {
    std::cout << "3^8: " << power_2(3, 8) << std::endl;
    return 0;
}
```


Logarithmic complexity

- Mathematical basis for $O(\log n)$ complexity
  - ✓ the time complexity is proportional to the number of times n must be divided by 2 until reaching 1
    - each division by 2, represents a reduction of the problem space by half
  - ✓ this process can be mathematically formulated as:

$$\frac{n}{2^k} \leq 1$$

where k is the number of divisions required to reduce the problem size to 1 or less

  - ✓ solving for k:

$$n \leq 2^k$$
$$\log_2 n \leq \log_2 2^k$$
$$\log_2 n \leq k$$

the smallest integer k satisfying this is $\lceil \log_2 n \rceil$

- Example
  - ✓ consider $n = 16$, the number of times 16 must be divided by 2 until reaching 1 is?

$$\frac{16}{2} = 8 \rightarrow \frac{8}{2} = 4 \rightarrow \frac{4}{2} = 2 \rightarrow \frac{2}{2} = 1$$

Linear vs logarithmic time

n	time (# days)	~ log(n) (# operations)
1	0.000	0
10	0.000	3
100	0.000	7
1000	0.000	10
10000	0.000	13
100000	0.000	17
1000000	0.000	20
10000000	0.000	23
100000000	0.000	27
1000000000	0.000	30
10000000000	0.000	33
100000000000	0.000	37
1000000000000	0.000	40
10000000000000	0.000	43
100000000000000	0.003	47
1000000000000000	0.028	50
10000000000000000	0.281	53
100000000000000000	2.809	56
1000000000000000000	28.086	60
10000000000000000000	280.863	63
100000000000000000000	2808.628	66

Intel Core i9-9900K	412,090 MIPS at 4.7 GHz

Practice

- Draw the recursion call tree
  - ✓ what can you tell about the computational cost?

```
// fibonacci sequence (recursive)
// 0 1 1 2 3 5 8 13 21 34 55 89 144 ...
uint64_t fibR(uint16_t n) {
    if (n < 2) {
        return n;
    } else {
        return fibR(n-1) + fibR(n-2);
    }
}

int main() {
    std::cout << fibR(5) << std::endl;
    return 0;
}
```